

Load Cell Management Unit (LCMU)

Hardware and Software Reference

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.2

Date: September 2026

This document describes the hardware, software architecture, CAN bus protocol, and load-cell/thrust measurement subsystems of the Load Cell Management Unit. The LCMU measures Ox load during oxidizer fill (in support of GSEMU's fill-to-target sequence) and measures thrust during test-stand burns, logging both to microSD.

Contents

1	Hardware Overview	2
1.1	Component Summary	2
1.2	I ² C Topology and Pin Map	2
1.3	Hardware Block Diagram	3
2	Software Architecture	3
2.1	Module Summary	4
2.2	Main Loop Control Flow	4
3	CAN Bus Protocol	5
3.1	Command Table	5
3.2	Fill Sequence (LCMU's Role)	5
4	Load Cell Subsystem	6
4.1	NAU7802 Configuration	6
4.2	Background Scan and Tare	6
5	Operating Modes	7
5.1	Idle Mode	7
5.2	Weighing Mode	7
5.3	Thrust Mode	7
6	Local UI (OLED + Buttons)	8
7	SD Logging	8
8	Real-Time Clock	9
9	Error Handling	9
10	Version History	9

1 Hardware Overview

1.1 Component Summary

Component	Part	Interface	Notes
Microcontroller	Adafruit Feather RP2040 Adalogger (dual-core Cortex M0+, 133 MHz, 8 MB QSPI flash, 264 KB RAM)	—	Onboard microSD, STEMMA QT
CAN Controller	Adafruit CAN Bus FeatherWing (MCP2515, TJA1051/3 transceiver)	SPI0	Link to EMU/GSEMU
OLED Display	Adafruit 128×64 FeatherWing (SH1107), 1.12", 3 buttons	I ² C	Landscape, rotated 180°
Real-Time Clock	Adafruit DS3231 Precision RTC, STEMMA QT	I ² C	Coin-cell (CR1220) backup
I ² C Multiplexer	Adafruit PCA9548, 8-channel STEMMA QT	I ² C	Load-cell ADCs on ports 3, 4, 7
Load Cell ADC (×3)	Adafruit NAU7802, 24-bit, STEMMA QT	I ² C (via mux)	One per load cell; fixed address, hence the mux
Load Cells (×3)	0–200 kgf each (0–600 kgf combined), tension/compression, 0.1% precision	—	3 m cable
SD Card	Onboard microSD (dedicated SPI1)	SPI1	
Battery	2S LiPo → external 5 V regulator	ADC (A0)	Resistor- divider sense

1.2 I²C Topology and Pin Map

The DS3231, PCA9548 mux, and all three NAU7802 boards share one STEMMA QT daisy chain:

Feather ↔ DS3231 (0x68) ↔ PCA9548 mux (0x70) ↔ ports 3/4/7 ↔ 3×NAU7802
(0x2A each)

The mux is required because all three NAU7802 boards share the same fixed I²C address (0x2A) — only one can be selected onto the shared bus at a time. Channel select is a direct two-byte I²C write (`Wire.beginTransmission(0x70); Wire.write(1 << channel); Wire.endTransmission()`), no dedicated mux library needed.

Pin / Signal	Value	Notes
CAN_CS_PIN	GPIO1 (RX)	RX/TX are reversed on this board versus the Feather M0 convention EMU/GSEMU use (confirmed against Adafruit's pinout page and the earlephilhower core's <code>pins_arduino.h</code>)
CAN_INT_PIN	GPIO13	Not read by firmware (this design polls <code>parsePacket()</code> from <code>loop()</code>); wired for consistency with EMU/GSEMU and left available for future interrupt-driven use
BUTTON_A_PIN	GPIO9	Arms thrust recording (bench testing)
BUTTON_B_PIN	GPIO6	Toggle weight recording
BUTTON_C_PIN	GPIO5	Tare
BATTERY_ADC_PIN	A0 (GPIO26)	Resistor-divider battery voltage sense, 12-bit ADC

1.3 Hardware Block Diagram

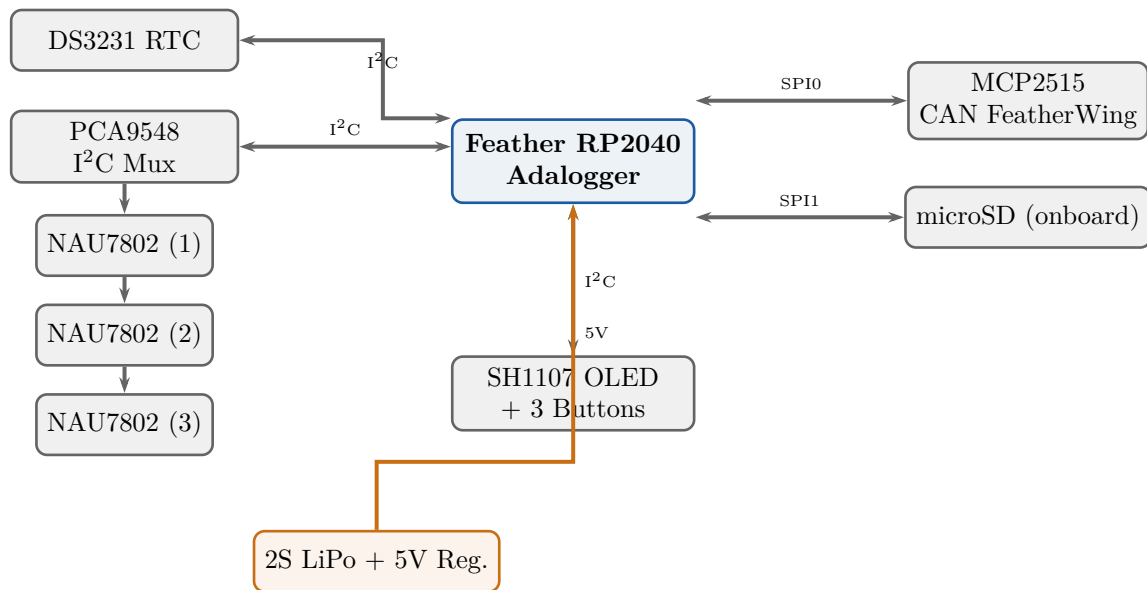


Figure 1: LCMU hardware block diagram.

2 Software Architecture

2.1 Module Summary

Module	Location	Responsibility
main.cpp	local	Setup, loop, CAN dispatch, recording modes, buttons, display refresh
KJO_Load_Cells	local	PCA9548 mux control, 3×NAU7802 background scan, tare, raw→lbf conversion
KJO_LCMU_Display	local	SH1107 OLED idle/run screen
KJO_LCMU_Files	local	SD file-sequencing (Find_Available_File())
KJO_Thrust_Buffer	local	Pre/post-trigger RAM ring buffer for high-rate thrust capture
KJO_CAN_Command_Defs	shared	CAN command codes, packet structs, node IDs
KJO_Status / KJO_Status_Display	shared	Boot-time scrolling status log (Report_Status())

2.2 Main Loop Control Flow

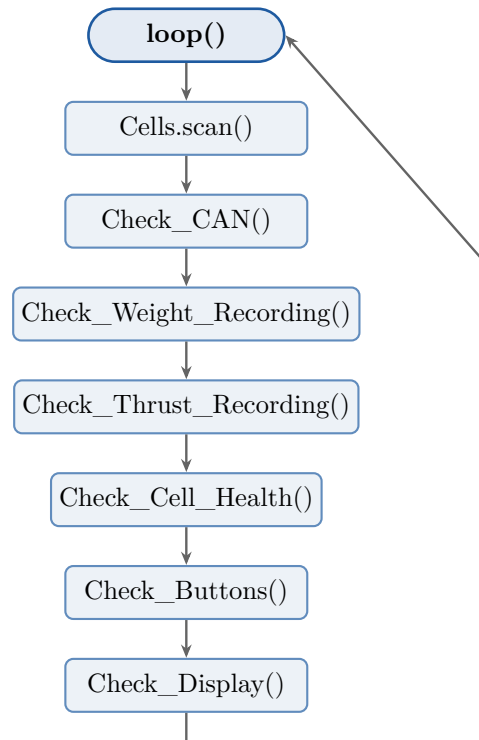


Figure 2: LCMU main loop control flow. `Cells.scan()` advances the background round-robin load-cell read by one step every iteration – everything else reads back whatever it most recently finished.

3 CAN Bus Protocol

Pure addressed command/response: every message is a request or a direct reply to one, no unsolicited broadcasts. Node IDs: EMU = 1, GSEMU = 2, LCMU = 3. CAN ID packing (11-bit standard frame): `CAN_ID = (destination << 4) | source`. Baud rate: 1 Mbps. Frame payload: `[command(1 byte)][params(up to 7 bytes)]`, defined in the shared `KJO_CAN_Command_Defs.h` header (separate from the LoRa-only `KJO_Command_Defs.h`, since the two transports serve different audiences).

3.1 Command Table

Command	Sent by	Response	Notes
<code>CAN_TARE</code>	EMU or GSEMU	ack	Tares all 3 channels (<code>Cells.tare()</code>)
<code>CAN_GET_BATTERY_VOLTAGE</code>	EMU	float volts	
<code>CAN_REPORT_CURRENT_WEIGHT</code>	EMU or GSEMU	float lbf	Combined 3-channel total; GSEMU polls this during an active fill
<code>CAN_BEGIN_WEIGHT_RECORDING</code>	EMU	ack/fail	Fails if Thrust mode is active
<code>CAN_STOP_WEIGHT_RECORDING</code>	EMU	ack	
<code>CAN_BEGIN_THRUST_RECORDING</code>	EMU	ack/fail	Arms threshold trigger; fails if Weighing mode is active
<code>CAN_STOP_THRUST_RECORDING</code>	EMU	ack	Manual override — forces immediate write-and-close
<code>CAN_GET_HEALTH_STATUS</code>	Any node	uint16 bitfield	SD/RTC/per-cell fault bits (see below)
<code>CAN_PING</code>	Any node	ack	Boot-time presence check only
<code>CAN_GET_CELL_RAW</code>	EMU	float (tare-adjusted raw)	<code>param</code> = channel index (0/1/2); diagnostic readout for <code>RCU_UNIT_TEST</code>

`CAN_GET_HEALTH_STATUS`'s response bitfield reuses bits 0–1 for SD/RTC init failure (shared convention across EMU/GSEMU/LCMU) and bits 8–10 for this board's own 3 load-cell channels (`HEALTH_LCMU_CELL_1_FAIL` through `_3_FAIL`).

3.2 Fill Sequence (LCMU's Role)

LCMU does not initiate or track the fill sequence itself — it only answers EMU's requests. EMU owns the whole session; GSEMU has no fill state machine of its own, it only opens/closes the Fill valve servo on command:

1. Operator commands **BEGIN_FILL** (RCU → EMU).

2. EMU (fresh start only — a resumed/paused session skips this, since re-taring mid-fill would zero the load cells against mass already flowed) sends **CAN_TARE** to LCMU.
3. EMU sends **CAN_OPEN_FILL_VALVE** to GSEMU.
4. EMU polls LCMU with **CAN_REPORT_CURRENT_WEIGHT** every 200 ms, comparing the tare-relative reading against a per-motor target EMU holds locally — never sent to GSEMU or LCMU.
5. Once the target is reached, EMU sends **CAN_CLOSE_FILL_VALVE** to GSEMU itself; no round trip through LCMU is needed to authorize the close.

4 Load Cell Subsystem

4.1 NAU7802 Configuration

Constant	Value	Unit	Meaning
LOAD_CELL_GAIN	128×	—	NAU7802 PGA gain
LOAD_CELL_EXCITATION_VOLTS	3.0	V	Bridge excitation, also the ADC reference (ratiometric)
LOAD_CELL_SAMPLE_RATE_SPS	320	SPS	Fastest available NAU7802 conversion rate
SAMPLE_AVERAGE_COUNT	9	readings	Averaged per reported sample; noise drops as $1/\sqrt{N}$
TARE_SAMPLE_COUNT	10	readings	Averaged per channel at tare time
LOAD_CELL_COUNTS_PER_LBF	2435.2	counts/lbf	Estimated (see derivation below); pending known-weight calibration

The excitation voltage is set below the NAU7802’s rated 2.4–4.0 V range: the STEMMA QT bus feeding all three boards runs off the Feather’s own 3.3 V rail (not the 5 V the Feather itself is powered from), leaving little headroom for the chip’s internal LDO to regulate cleanly above 3.0 V. Reaching the full rated range for better SNR would require rewiring the NAU7802 boards’ power pin to a 5 V source instead of the 3.3 V STEMMA QT bus — a hardware change, deferred.

LOAD_CELL_COUNTS_PER_LBF is derived from the load cell’s rated output (1 mV/V at full-scale 200 kgf) and the NAU7802’s ratiometric ADC transfer function:

$$\text{full-scale ADC code} = \frac{V_{\text{diff}} \times \text{Gain}}{V_{\text{ref}}} \times 2^{23} = \frac{0.003 \times 128}{3.0} \times 8388608 \approx 1,073,742 \text{ counts}$$

$$\text{full-scale lbf} = 200 \text{ kgf} \times 2.20462 \approx 440.92 \text{ lbf} \quad \Rightarrow \quad \text{counts/lbf} = \frac{1,073,742}{440.92} \approx 2435.2$$

This is an estimate pending an actual known-weight calibration pass.

4.2 Background Scan and Tare

`Load_Cells::scan()`, called unconditionally every `loop()` iteration, advances a round-robin background read: one raw attempt per call for whichever channel is currently up, finalizing that channel’s cached average once `SAMPLE_AVERAGE_COUNT` attempts have accumulated before moving to the next channel. Every other reader (`readChannels()`, `readWeight()`, `rawCached()`) is instant — it reads back the most recently completed batch rather than triggering a fresh ADC read. A channel reports healthy if at least one reading in its most recent batch succeeded.

`tareChannel()` averages `TARE_SAMPLE_COUNT` raw readings back-to-back and stores the result as that channel's offset in raw ADC counts (not lbf) — this keeps the tare valid independent of `LOAD_CELL_COUNTS_PER_LBF`, so a later calibration revision doesn't invalidate an existing tare. `tare()` tares all 3 channels in turn and is run automatically as the last step of `setup()`, so the unit starts up already zeroed, as well as on `CAN_TARE` or Button C.

5 Operating Modes

`current_mode` (`MODE_IDLE` / `MODE_WEIGHING` / `MODE_THRUST`) gates both recording behavior and which OLED buttons are watched. Weighing and Thrust are mutually exclusive — a `CAN_BEGIN_THRUST_RECORDING` while weighing (or vice versa) is rejected with `ack = fail`.

5.1 Idle Mode

Continuously samples all 3 load cells via the background scan, answers on-demand CAN queries, and watches all three OLED buttons.

5.2 Weighing Mode

Opens the next sequenced `Weight_N.txt` and appends one timestamped sample (raw counts and lbf, all 3 channels plus the combined total) every `WEIGHT_SAMPLE_INTERVAL_MS` (1 s), until `CAN_STOP_WEIGHT_RECORDING` or Button B closes the file. Only Button B is watched while weighing; A and C are ignored.

5.3 Thrust Mode

Constant	Meaning
<code>THRUST_ARM_THRESHOLD_LBF</code>	1.0 lbf — sustained thrust above this triggers capture
<code>THRUST_PRE_TRIGGER_MS</code>	1000 ms of pre-trigger data buffered continuously while armed
<code>THRUST_SAMPLE_INTERVAL_MS</code>	100 ms (10 Hz) — same averaged read path as Weighing mode
<code>THRUST_POST_TRIGGER_DURATION_MS</code>	120,000 ms (120 s) — fixed once triggered, regardless of the thrust profile in between
<code>THRUST_NEVER_TRIGGERED_TIMEOUT_MS</code>	21,000 ms (21 s) — independent safety net so an armed-but-untriggered bench test (no real force applied) still ends on its own

`CAN_BEGIN_THRUST_RECORDING` (or Button A, bench testing) opens the next sequenced `Thrust_N.txt` and arms `Thrust_Buffer`, which continuously buffers the trailing `THRUST_PRE_TRIGGER_MS` of samples in RAM. Once thrust crosses `THRUST_ARM_THRESHOLD_LBF`, the buffer freezes and writes that pre-trigger snapshot, then continues sampling into a second linear (non-overwriting) buffer for the fixed post-trigger duration. If armed but never triggered, the *never-triggered timeout* completes the recording anyway — without it, a bench test with no real force applied would never end, since the post-trigger completion logic only starts counting once a trigger has actually occurred. No button is watched while in Thrust mode; it only ends via one of these two timers or `CAN_STOP_THRUST_RECORDING`'s manual override.

Both timers were revised from the original design during implementation: real hardware testing found a sustained-low-thrust auto-cutoff (the original design) risked ending a real burn’s recording early on a transient thrust dip mid-burn, and could never run at all for a bench test that never crosses the arm threshold in the first place. The two-independent-timer approach above replaced it.

6 Local UI (OLED + Buttons)

The SH1107 panel is mounted rotated 180° from its natural orientation. The idle/run screen has 3 fixed lines: the top line shows the 3 individual load-cell readings in fixed-width fields (left to right = cells 1, 2, 3); the middle line shows the combined weight/thrust total, large, bold, and centered in a fixed-width field so the decimal point never shifts, followed by a small gap and a normal-weight **lbf** suffix; the bottom line shows battery voltage right-justified, with the remaining left-hand space showing a recording/tare status label when applicable.

Button	Action
A	Idle only. Arms thrust recording (bench testing) — one-shot; the recording ends on its own (see Thrust Mode timers above), not via a second press.
B	Idle: starts weight recording. Weighing: stops it. Ignored while Thrust recording.
C	Idle only. Tare — see the deliberate on-screen sequence below.

Tare sequence (Button C, or automatically as the last step of `setup()`): shows “Tare” for 1 s, then for each of the 3 cells in turn, averages `TARE_SAMPLE_COUNT` readings, shows “<N>: <raw average>” on the bottom line for 1 s, then moves to the next cell. Triggered on press (not release) — the sequence has its own fixed timing regardless of how long the button is physically held.

Note

An earlier revision let Button C double as a thrust-stop control, which made the tare sequence look broken/inconsistent whenever it fired mid- thrust-test. Button C is now exclusively the tare button, and no button is watched at all during Thrust mode.

7 SD Logging

Three independent, sequentially-numbered file streams on the SD root (`Find_Available_File()` scans for the lowest unused index):

File	Contents
<code>Log_N.txt</code>	General timestamped event log, same convention as EMU/GSEMU/RCU. One file per power cycle, opened at boot.
<code>Weight_N.txt</code>	Opened on <code>CAN_BEGIN_WEIGHT_RECORDING</code> ; one row per 1 Hz sample; closed on stop.
<code>Thrust_N.txt</code>	Opened on <code>CAN_BEGIN_THRUST_RECORDING</code> (arm); written in the two-stage burst described above; closed at auto-cutoff or manual stop.

Each data file (`Weight_`/`Thrust_`) opens with a header recording gain, sample rate, excitation voltage, the averaging count, the current `LOAD_CELL_COUNTS_PER_LBF` value, and each channel’s tare offset in raw counts — captured at file-open time rather than referenced as a constant, since

tare offsets change every tare cycle. This lets the lbf columns be independently reconstructed later even if calibration is subsequently revised. Both raw counts and converted lbf values are logged per row for the same reason.

Each mode gets a fresh index the next time it starts, so multiple weighings/burns in one power cycle don't overwrite each other.

8 Real-Time Clock

DS3231, via `RTClib` (the same library RCU uses). On `begin()`, if `lostPower()` reports true, the clock is set from compile time (`__DATE__`/`__TIME__`) — the same auto-heal pattern EMU's PCF8523 uses.

9 Error Handling

- **I²C failures** (DS3231, mux, or any individual NAU7802): logged to `Log_N.txt` and shown on the OLED boot status; LCMU continues in a degraded state (computing weight/thrust from whichever channels still respond) rather than halting.
- **CAN failures:** `CAN_Controller.begin()` failure at boot is logged/shown; LCMU continues local operation (buttons, SD logging) without a working bus link.
- **SD failures:** a failed `SD.begin()` disables file logging but doesn't halt the unit; a failed `SD.open()` for the current mode logs a write-fail message and that recording attempt simply fails (ack = fail over CAN) rather than losing buffered data silently.

10 Version History

Version	Date	Notes
0.1	September 2026	Initial release.
0.2	September 2026	Corrected the Fill Sequence subsection: EMU owns the fill session directly (<code>CAN_TARE</code> and <code>CAN_REPORT_CURRENT_WEIGHT</code> sent by EMU itself, target held locally on EMU), not GSEMU as the initial draft stated – that description matched an earlier, superseded design.